

📖 MASTER SECURITY DOCUMENTATION - Atta Chakki App

> ****Complete Security Guide****: All-in-one document covering threats, protections, compliance, and implementation for the offline-first Atta Chakki Management App (React Native/Expo with SQLite).

📑 Table of Contents

1. [Security Overview & Architecture] (#1-security-overview--architecture)
2. [Threat Model (STRIDE Analysis)] (#2-threat-model-stride-analysis)
3. [Data Security & Protection] (#3-data-security--protection)
4. [Authentication & Access Control] (#4-authentication--access-control)
5. [Code & Mobile-Specific Security] (#5-code--mobile-specific-security)
6. [API Security (Cloud Sync - Phase 2)] (#6-api-security-cloud-sync--phase-2)
7. [Backup & Disaster Recovery] (#7-backup--disaster-recovery)
8. [Incident Response Plan] (#8-incident-response-plan)
9. [Compliance & Legal] (#9-compliance--legal)
10. [Security Testing & Penetration] (#10-security-testing--penetration)
11. [Pre-Release Security Checklist] (#11-pre-release-security-checklist)

1. Security Overview & Architecture

🎯 Assets (What to Protect)

Asset	Value	Owner	Primary Threat
----- ----- ----- -----			
Customer PII (name, phone, address) HIGH Customers Privacy breach, identity theft			
Financial Records (payments, udhar) CRITICAL Owner Money loss, fraud			
Business Data (sales, inventory) MEDIUM Owner Competitive disadvantage			
SMS Capability MEDIUM Owner Spam, reputation damage			
App Authentication HIGH Owner Unauthorized access			
Encryption Keys CRITICAL System Complete data exposure			

📊 Threat Landscape (Priority-wise)

Threat	Likelihood	Impact	Priority
----- ----- ----- -----			

Phone theft/loss	● HIGH	● CRITICAL	P0
Unauthorized local access	● MEDIUM	● HIGH	P0
Data corruption	● MEDIUM	● MEDIUM	P1
Malicious app modification	● LOW	● HIGH	P1
Cloud sync interception	● LOW	● HIGH	P2
SMS spoofing	● VERY LOW	● MEDIUM	P3

🛡️ Defense-in-Depth Strategy

```mermaid

graph TD

```

A[Physical Security] -->|Device PIN/Pattern| B[OS Security Layer]
B -->|App Lock| C[Application Layer]
C -->|Encryption| D[Data Layer - SQLite]
C -->|HTTPS| E[Network Layer]
E -->|TLS 1.3| F[Cloud Backend]

D -->|Encrypted Backup| G[Local Storage]
D -->|Encrypted Sync| F

H[Security Monitoring] -.->|Audit Logs| C
H -.->|Integrity Checks| D
H -.->|Network Inspection| E

```

## 2. Threat Model (STRIDE Analysis)

### S - Spoofing (Identity Theft)

- **App impersonation:** Fake app on Play Store. *Mitigation:* Unique package name, verify developer signature.
- **User impersonation:** Employee/family accesses owner's phone. *Mitigation:* App-level PIN/biometric lock.
- **SMS spoofing:** Fake SMS from owner's number. *Mitigation:* Move to WhatsApp in Phase 2.

### T - Tampering (Data Modification)

- **Database tampering:** Root user edits SQLite file. *Mitigation:* SQLCipher encryption.
- **Code tampering:** Decompile APK. *Mitigation:* ProGuard obfuscation + integrity checks.
- **Backup tampering:** Edit backup file. *Mitigation:* HMAC signature on backups.

### R - Repudiation (Deny Actions)

- **Deny entry/payment:** Customer/Operator denies actions. *Mitigation:* SMS confirmation + SQLite Audit logs table with timestamps.

## I - Information Disclosure (Data Leak)

- **Unencrypted backup leak:** File shared externally. *Mitigation:* Encrypt backups.
- **Screenshot leak:** Visual privacy breach. *Mitigation:* Prevent screenshots on sensitive screens (expo-screen-capture).

## D - Denial of Service

- **Database corruption:** App crash during write. *Mitigation:* Write-ahead logging (WAL), SQL transactions.
- **Storage full:** Phone storage maxed out. *Mitigation:* Auto-archive old data.

## E - Elevation of Privilege

- **Root privilege:** Rooted device bypasses security. *Mitigation:* Detect root (jail-monkey), refuse to run.
- **Operator deleting entries:** *Mitigation:* Role-Based Access Control (RBAC).

# 3. Data Security & Protection

## Data at Rest (Local Database)

Uses **SQLCipher** for database-level encryption. The key is securely generated and stored in the OS Keystore/Keychain.

```
import * as SQLite from 'expo-sqlite';
import * as SecureStore from 'expo-secure-store';
import * as Crypto from 'expo-crypto';

// Generate/Get encryption key
async function getOrCreateDBKey() {
 let key = await SecureStore.getItemAsync('db_encryption_key');
 if (!key) {
 key = await Crypto.getRandomBytesAsync(32).then(bytes =>
bytes.toString('base64'));
 await SecureStore.setItemAsync('db_encryption_key', key);
 }
 return key;
}

// Open secure DB
async function openSecureDB() {
 const key = await getOrCreateDBKey();
 const db = SQLite.openDatabase('chakki.db');
 await db.execAsync(`PRAGMA key = '${key}';`);
 await db.execAsync('PRAGMA journal_mode = WAL;'); // Crash safety
 return db;
}
```

```
}
```

## Sensitive Field Encryption

For highly sensitive fields like customer phone numbers (to prevent plaintext exposure even if DB structure is leaked):

```
async function encryptField(plaintext) {
 const key = await getOrCreateDBKey();
 return await
Crypto.digestStringAsync(Crypto.CryptoDigestAlgorithm.SHA256,
plaintext + key);
}
```

## 4. Authentication & Access Control

### App-Level Lock (Biometric + PIN)

```
import React, { useState, useEffect } from 'react';
import * as LocalAuthentication from 'expo-local-authentication';
import * as SecureStore from 'expo-secure-store';

export default function AppLock({ children }) {
 const [isUnlocked, setIsUnlocked] = useState(false);

 useEffect(() => { checkAuth(); }, []);

 async function checkAuth() {
 const lockEnabled = await
SecureStore.getItemAsync('app_lock_enabled');
 if (lockEnabled !== 'true') return setIsUnlocked(true);

 const hasHardware = await LocalAuthentication.hasHardwareAsync();
 const isEnrolled = await LocalAuthentication.isEnrolledAsync();

 if (hasHardware && isEnrolled) {
 const result = await LocalAuthentication.authenticateAsync({
 promptMessage: 'App खोलने के लिए',
 fallbackLabel: 'PIN दर्ज करें',
 });
 if (result.success) return setIsUnlocked(true);
 }
 // Fallback to custom PIN screen UI
 }

 if (!isUnlocked) return <LockScreen onUnlock="{ () }">
```

```

setIsUnlocked(true)} />;
 return children;
}

```

## Role-Based Access Control (Multi-User)

```

const ROLES = { OWNER: 'owner', OPERATOR: 'operator', VIEWER: 'viewer'
};

function usePermission(action) {
 const user = useCurrentUser();
 const permissions = {
 [ROLES.OWNER]: ['read', 'create', 'update', 'delete',
'manage_users'],
 [ROLES.OPERATOR]: ['read', 'create', 'update'],
 [ROLES.VIEWER]: ['read'],
 };
 return permissions[user.role]?.includes(action) ?? false;
}

```

# 5. Code & Mobile-Specific Security

## 1. Root & Jailbreak Detection

```

import * as Device from 'expo-device';
import JailMonkey from 'jail-monkey';

async function checkDeviceSecurity() {
 if (JailMonkey.isJailBroken()) {
 Alert.alert('⚠ Security Risk', 'App rooted/jailbroken device pe
safely nahi chal sakta');
 return false;
 }
 return true;
}

```

## 2. Prevent Screenshots (Android)

```

import { preventScreenCapture, allowScreenCapture } from
'expo-screen-capture';

useEffect(() => {
 preventScreenCapture(); // Blocks screenshots on sensitive screens
 return () => allowScreenCapture();
}

```

```
}, []);
```

### 3. Input Validation & SQL Injection Prevention

Always use parameterized queries, never template literals.

```
// ✅ SAFE - Parameterized queries
db.getAllAsync('SELECT * FROM customers WHERE name = ?', [name]);
```

## 6. API Security (Cloud Sync - Phase 2)

### HTTPS and Supabase Security

```
import { createClient } from '@supabase/supabase-js';

const supabase =
 createClient(' [https://your-project.supabase.co] (https://your-project.
supabase.co)', 'your-anon-key', {
 auth: { persistSession: true, autoRefreshToken: true },
});

// Row-Level Security (RLS) in Supabase SQL:
// CREATE POLICY "Users access own data" ON customers FOR ALL USING
(auth.uid() = owner_id);
```

### API Rate Limiting (Client-Side Throttle)

```
import pThrottle from 'p-throttle';
const throttledSync = pThrottle({ limit: 10, interval: 1000
})(syncCustomer);
```

## 7. Backup & Disaster Recovery

### Backup Strategy

- **Local Backups:** Stored safely as Base64 encoded files with HMAC signatures.
- **Recovery Flow:** User inputs backup\_key -> Verifies HMAC -> Restores chakki.db.

```
import * as FileSystem from 'expo-file-system';
import * as Crypto from 'expo-crypto';

async function createEncryptedBackup() {
 const dbPath = `${FileSystem.documentDirectory}SQLite/chakki.db`;
 const dbData = await FileSystem.readAsAsStringAsync(dbPath, {
 encoding: FileSystem.EncodingType.Base64 });
```

```

const backupKey = await SecureStore.getItemAsync('backup_key');
const signature = await
Crypto.digestStringAsync(Crypto.CryptoDigestAlgorithm.SHA256, dbData +
backupKey);

const backup = { version: '1.0', timestamp: new
Date().toISOString(), data: dbData, signature };
// Save and share via WhatsApp/Drive
}

```

## 8. Incident Response Plan

### Incident Classification

| Severity | Example                              | Response Target |
|----------|--------------------------------------|-----------------|
| CRITICAL | Database leaked / API breached       | 1 hour          |
| HIGH     | Unauthorized app access              | 4 hours         |
| MEDIUM   | Multiple failed PINs / Root detected | 24 hours        |

### Response Playbook (If Breach Detected)

1. **Isolate:** Disable cloud sync & kill affected sessions.
2. **Assess:** Query security\_logs to determine data affected.
3. **Patch:** Push hotfix via Over-The-Air (OTA) updates using expo-updates.
4. **Rotate:** Change Supabase API keys & JWT secrets.

## 9. Compliance & Legal

### Applicable Regulations

- **IT Act 2000 (Section 43A):** Compensation for failure to protect data.
- **DPDP Act (India):** Strict guidelines on data minimization, user consent, and purpose limitation.
- **TRAI DLT:** Required for SMS sending features to prevent spam blocks.

### Privacy Policy Requirements

- **What:** Phone numbers, names, financial transaction history.
- **Why:** Core functioning of the ledger (Udhar) and SMS receipts.
- **How:** Local-first, encrypted SQLite database. Cloud sync only via explicit user consent.
- **Rights:** User can export data (JSON backup) or trigger secure account deletion.

## 10. Security Testing & Penetration

### Testing Methodology (OWASP Mobile Top 10)

- **Static Analysis:** npm audit, react-native-devsecops analyze.
- **Dynamic Analysis:** Install APK and test with objection & Frida to ensure SecureStore and SQLCipher cannot be trivially bypassed on rooted devices.
- **Network:** Proxy through Burp Suite to verify TLS 1.3 and ensure no cleartext data leaks.

## 11. Pre-Release Security Checklist

### Code & Environment

- ☐ No hardcoded secrets (API keys strictly in .env).
- ☐ Dependencies audited (npm audit fix --production).
- ☐ Code obfuscation enabled (ProGuard configured in build.gradle).
- ☐ Debug modes disabled in production packaging.

### Data Security

- ☐ Database encrypted via SQLCipher.
- ☐ Backups exported with HMAC signatures.
- ☐ SecureStore used for all keys instead of AsyncStorage.
- ☐ No sensitive PII or passwords printed in console.log.

### App Security

- ☐ App lock (Biometric/PIN) active.
- ☐ Root/Jailbreak detection enabled.
- ☐ Screenshot prevention active on ledger screens.
- ☐ Parameterized queries used for all DB interactions.